

cmc_lab6

October 1, 2026

1 Lab 6 - Charlie Caravati

```
[17]: import numpy as np
import math
import time
from numpy.linalg import inv
from numpy.linalg import norm
```

```
[49]: def evalF(x):
    F = np.array([
        4*x[0]**2 + x[1]**2 - 4,
        x[0] + x[1] - np.sin(x[0]-x[1])
    ])

    return F

def evalJ(x):

    J = np.array([
        [8 * x[0],          2.0 * x[1]],
        [1 - np.cos(x[0]-x[1]), 1 + np.cos(x[0] - x[1])]
    ])

    return J
```

```
[19]: def newton(x0,tol,Nmax):

    ''' inputs: x0 = initial guess, tol = tolerance, Nmax = max its'''
    ''' Outputs: xstar= approx root, ier = error message, its = num its'''

    for its in range(Nmax):
        J = evalJ(x0)
        Jinv = inv(J)
        F = evalF(x0)
```

```

    x1 = x0 - Jinv.dot(F)

    if (norm(x1-x0) < tol):
        xstar = x1
        ier = 0
        return[xstar, ier, its]

    x0 = x1

xstar = x1
ier = 1
return[xstar, ier, its]

```

```

[20]: def lazynewton(x0,tol,Nmax):

    ''' Lazy Newton = use only the inverse of the Jacobian for initial guess'''
    ''' inputs: x0 = initial guess, tol = tolerance, Nmax = max its'''
    ''' Outputs: xstar= approx root, ier = error message, its = num its'''

    J = evalJ(x0)
    Jinv = inv(J)
    for its in range(Nmax):

        F = evalF(x0)
        x1 = x0 - Jinv.dot(F)

        if (norm(x1-x0) < tol):
            xstar = x1
            ier = 0
            return[xstar, ier, its]

        x0 = x1

    xstar = x1
    ier = 1
    return[xstar, ier, its]

```

```

[67]: def slackernewton(x0,tol,Nmax):

    J = evalJ(x0)
    Jinv = inv(J)
    for its in range(Nmax):

        F = evalF(x0)
        x1 = x0 - Jinv.dot(F)

        if (norm(x1-x0) < tol):

```

```

        xstar = x1
        ier = 0
        return[xstar, ier, its]

    x0 = x1

    if np remainder(its+1, 10) == 0:
        J = evalJ(x0)
        Jinv = inv(J)

    xstar = x1
    ier = 1
    return[xstar, ier, its]

```

`np remainder(0, 10)` evaluates to 0 when `its = 0`, which is where it starts

```
[21]: root, er, iters = newton([2, 0.5], 10e-6, 10000)
```

```
print(f"{root}, {er}, {iters}")
```

```
[1. 1.], 0, 5
```

```
[22]: root, er, iters = newton([3, 5], 10e-6, 10000)
```

```
print(f"{root}, {er}, {iters}")
```

```
[3.51286242 7.43987744], 1, 9999
```

```
[72]: root, er, iters = lazynewton([1, 0], 10e-10, 10000)
```

```
print(f"{root}, {er}, {iters}")
```

```
[ 0.99860694 -0.10553049], 0, 6
```

```
[71]: root, er, iters = slackernewton([1, 0], 10e-10, 10000)
```

```
print(f"{root}, {er}, {iters}")
```

```
[ 0.99860694 -0.10553049], 0, 6
```